

数据结构机考学习1：顺序表

我们数据结构下周五要机考，
需要学习一下c++类和模板的语法，
要看得懂题目给的是什么。
而我在蓝桥杯用的都是已经封装好的函数，
因此这几天需要学习一下这部分的知识。
下面写一下小总结：

一：基础语法：

1.两种模板参数

```
template<typename T, int n = 100>
class seqlist {
private:
    T data[n];
    int length;
```

参数	类型	说明
typename T	类型参数	代表一个数据类型，可以是 int、double、string 等
int n	非类型参数	代表一个具体的整数值，在编译时确定

2.私有成员

```
private:
    T data[N];        // 存储数据的静态数组
    int length;       // 当前线性表的实际长度
};
```

二.基础操作：

由于之前已经有了基础知识，

下面的几个基础操作我将不再赘述。

1.构造

```
seqlist() {  
    length = 0;  
}  
seqlist(T a[], int n) {  
    for (int i = 0; i < n; i++) {  
        data[i] = a[i];  
    }  
    length = n;  
}
```

2.求长度和求位置的值

```
int getlength() {  
    return length;  
}  
T get(int i) {  
    return data[i - 1];  
}
```

3.求值所在的位置

```
int locate(T x) {  
    for (int i = 0; i < length; i++) {  
        if (data[i] == x) {  
            return i + 1;  
        }  
    }  
    return 0;  
}
```

4.在指定位置插入

```

void insert(int i, T x) {
    for (int j = length; j >= i; j--) {
        data[j] = data[j - 1];
    }
    data[i - 1] = x;
    length++;
}

```

5.删除指定位置的元素

```

T deleted(int i) {
    T k = data[i - 1];
    for (int j = i; j < length; j++) {
        data[j - 1] = data[j];
    }
    length--;
    return k;
}

```

6.打印和更改指定位置的元素

```

void print() {
    for (int i = 0; i < length; i++) {
        cout << data[i] << " ";
    }
    cout << endl;
}

void set(int i, T x) {
    data[i - 1] = x;
}

```

有了以上的基础操作后，变可以实现其它功能。

而这些功能便是考题。

三.例题

由于这次机考的题目会比较简单，

因此我选的例题会比较简单。

题目1：顺序表的逆置

把 [1,2,3,4,5] 变成 [5,4,3,2,1]

这题比较简单，简单说一下思路：

方法：**双指针交换**

- 左指针 i ，右指针 j
- 交换 → 两边同时向中间移动

代码如下：

```
void reverse(seqlist<T, n>& list) {
    int i = 0;
    int j = list.getlength() - 1;
    while (i < j) {
        T temp = list.get(i + 1);
        list.set(i + 1, list.get(j + 1));
        list.set(j + 1, temp);
        i++;
        j--;
    }
}
```

注意：

i 表示的是数组的下标，

而get函数和set函数的参数是顺序表的位置，

而两者的关系是相差1.

题目2：删除顺序表中所有值为 x 的元素

[1,2,3,2,4,2,5] 删掉所有 2 → [1,3,4,5]

思路：**双指针覆盖法**

- slow：新表的下标
- fast：遍历原表

比较简单，不再赘述。

代码如下：

```

template<typename T, int n>
void deleteAll(seqlist<T, n>& list, T x) {
    int slow = 0;
    int len = list.getLength();

    for (int fast = 0; fast < len; fast++) {
        T val = list.get(fast + 1);
        if (val != x) {
            list.set(slow + 1, val);
            slow++;
        }
    }
}

```

四.课件补充内容和题目

有了这两道例题的引入，对顺序表就有了基本的了解。

接着我看了一下我们课件上的一些内容，可能会成为考题，

便顺便一起看完了，总结一些感觉有用的。

1.循环移位

方法一：逐次移动

思路很简单，不再赘述。

直接看代码就好了。

先写循环左移1位

```

template<typename T, int n>
void leftRotateOne(seqlist<T, n>& list) {
    T first = list.get(1); // 保存第一个元素
    for (int i = 2; i <= list.getLength(); i++) {
        list.set(i - 1, list.get(i)); // 左移
    }
    list.set(list.getLength(), first); // 第一个元素放到末尾
}

```

再写循环左移k位（调用k次）

```
template<typename T, int n>
void leftRotate(seqList<T, n>& list, int k) {
    k = k % list.getLength(); // 处理k大于长度的情况
    for (int i = 0; i < k; i++) {
        leftRotateOne(list);
    }
}
```

方法二：三次反转法

这里以左移k位举例

公式：

循环左移 k 位 = 反转(1, k) + 反转(k+1, n) + 反转(1, n)

其中 反转(a, b) 表示反转从位置 a 到位置 b 的元素（位置从1开始计数）

三次反转巧妙地实现了这个效果：

步骤	操作	效果
①	反转前k个	前k个元素内部顺序颠倒
②	反转后n-k个	后n-k个元素内部顺序颠倒
③	反转整体	所有元素位置正确

代码在例题中已经实现过reverse了，这里不再赘述。

```
// 反转顺序表中 [start, end] 区间的元素（位置从1开始）
template<typename T, int n>
void reverseRange(seqList<T, n>& list, int start, int end) {
    while (start < end) {
        T temp = list.get(start);
        list.set(start, list.get(end));
        list.set(end, temp);
        start++;
        end--;
    }
}
```

循环左移k位：三次反转

```

template<typename T, int n>
void leftRotate(seqList<T, n>& list, int k) {
    int len = list.getLength();
    k = k % len; if (k == 0) return;
    reverseRange(list, 1, k); // 步骤1: 反转前k个元素
    reverseRange(list, k + 1, len); // 步骤2: 反转后n-k个元素
    reverseRange(list, 1, len); // 步骤3: 反转整个表
}

```

操作	口诀	反转顺序
左移k位	前、后、整	①前k ②后n-k ③整体
右移k位	整、前、后	①整体 ②前k ③后n-k

2.有序表插入

思路：

1. 找到第一个大于 x 的位置
2. 在该位置插入

比较简单，不再赘述。

```

void insertOrdered(seqList<T, n>& list, T x) {
    int len = list.getLength();
    // 步骤1: 找到插入位置
    int i = 1;
    while (i <= len && list.get(i) < x) {
        i++;
    }
    // 步骤2: 在第i个位置插入
    list.insert(i, x);
}

```